

SEMINAR · 2026.07

바이브 코딩이란 무엇인가?

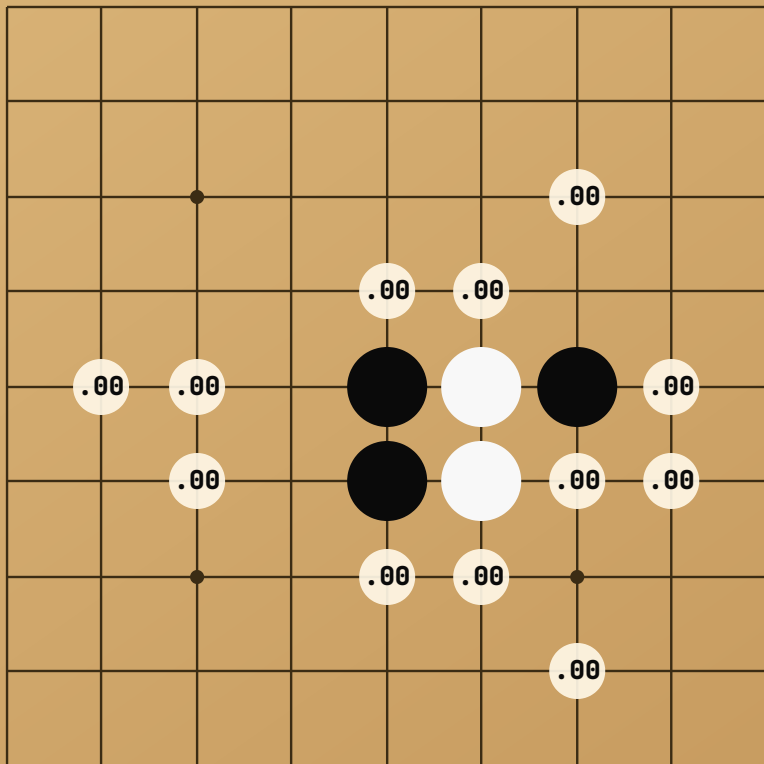
AI란 무엇이고, AI로 생산성을 올린다는 건 무슨 뜻일까? (feat. vibe coding)

원작 · syngcha.com/vibe-coding-2605 — 이 자료를 바탕으로 재구성·수정

CHAPTER 0 · HOOK

AI가 무엇일까요?

먼저, 알파고 이야기로 시작하죠.



바둑판 위에 $19 \times 19 = 361$ 개의 좌표가 있습니다.

알파고는 매 수마다 묻습니다.

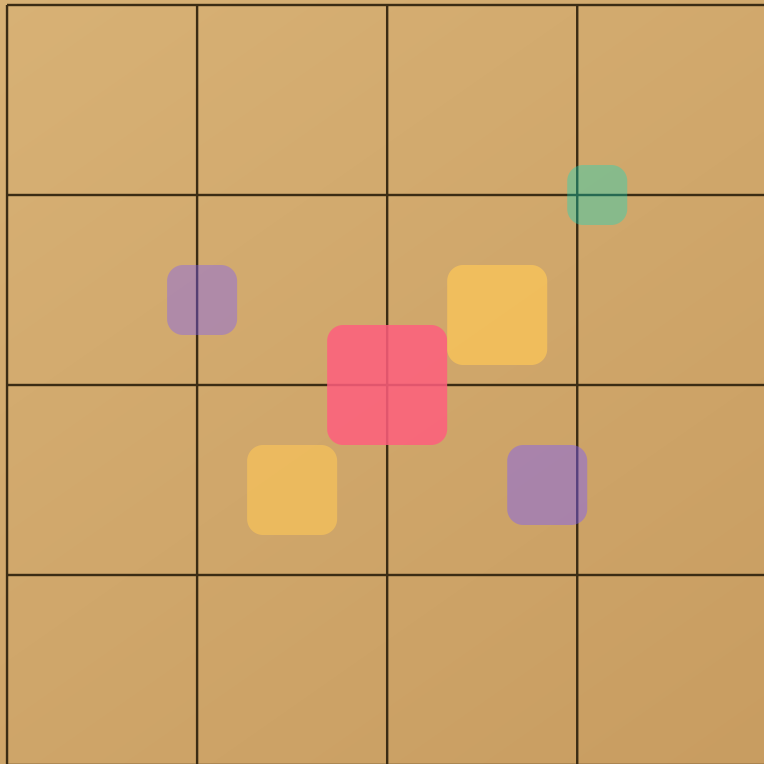
"여기 두면 이길 확률이 얼마야?"

361칸 전부에 대해 답을 계산합니다.

그리고 가장 **승률이 높은 칸**에 둡니다.

* 그림은 설명을 위해 9×9로 축약한 바둑판입니다.

그러니까 알파고가 하는 일은...



probability heatmap

"수를 둔다"가 아니라

"확률 분포를 그린다"

그것도 여러 수 앞을 내다본 뒤 — 가장 빨강게 빛나는 곳에 돌을 놓을 뿐입니다.

알파고 = 확률 계산기

그런데 알파고는 왜 그게 되죠?

데이터

지금까지 인류가 둔 수십만 판의 바둑 기보.

학습

"이런 모양일 때 → 이긴 사람은 다음에 어디에 뒀는가"를 통계로 배움. 그 뒤엔 자기 자신과 수백만 판을 더 두며 다듬음.

결국, "역사상 가장 승률이 높았던 다음 수"를 추천하는 기계.

이건 알파고만의 이야기가 아닙니다.

자율주행

카메라 영상 + 차량 상태 →

"다음 1초 동안 어떻게 움직일까"의 확률 분포를
그린다.

알파고

바둑판 상태 →

"다음 수는 어디일까"의 확률 분포를 그린다.

ChatGPT

지금까지의 문맥 →

"다음 단어는 무엇일까"의 확률 분포를 그린다.

형태만 다를 뿐, **전부 같은 로직**입니다.

그럼 우리가 매일 쓰는 LLM은?

극단적으로 단순화하면 —

"지금까지의 글자들을 보고, 다음에 올 단어를 예측하는 확률 계산기"

그게 전부입니다. 진심으로요.

예를 들어볼까요?

prompt: "오늘 날씨가 정말 "



한 단어 뽑고 → 다시 prompt에 넣고 → 또 뽑고... 반복하면 우리가 보는 "긴 문장"이 됩니다.

그래서 **AI 3대 석학, 안 르쿤**은 LLM을 까쥔다.

"세상을 이해하지 못한 채,
다음 단어만 그럴듯하게 뽑는 기계다.
이 길로는 인간 수준의 지능에 도달할 수 없다."

— Yann LeCun, 前 Meta Chief AI Scientist · 튜링상 수상자 (요지)

그리고 이 비판은 — 기술적으로는 맞습니다.

하지만.

그렇다고 LLM이 의미가 없는 건 아닙니다.

다시 알파고로 돌아가 보면...

알파고

"다음 수의 승률"을
계산할 수 있는 이유

지금까지 인류가 둔 수십만 판의 기보가 있기 때문.

LLM

"다음 단어의 확률"을
계산할 수 있는 이유

인류가 써온 수십조 단어의 글이 있기 때문.

둘 다 결국, "역사상 가장 승률 높았던 답"을 찾는 기계.

그리고 요즘은 "언어"의 정의가 더 넓어졌죠.



텍스트



코드



이미지



영상 / 음악

이미지도, 영상도, 음악도 — 결국 "순서가 있는 데이터"로 바꾸면
똑같이 "역사상 가장 그럴듯한 다음"을 찾는 문제가 됩니다.

그럼 이걸 **한마디로** 하면 — AI는 대체 무엇일까요?

저는 AI를 이렇게 이해합니다.

Q: "이번 분기 매출, 어떻게 올리지?"

HISTORY · 인류가 시도해온 모든 방법

신규 채널 확대
.00

시즌 할인
.00

고객 세분화
.00

리마케팅 광고
.00

구독 전환
.00

가격 인하
.00

인플루언서 협업
.00

재구매 유도
.00

추천인 제도
.00

번들 상품
.00

후기 이벤트
.00

재고 소진 세일
.00

역사상 가장 승률 높은 답을 찾는 "지능형 도서관"

이 도서관과 끊임없이 대화하며 답을 찾아가는 것 — 그 방식에 이름을 붙이면 "바이브 코딩"입니다.

그럼 "바이브 코딩", 구체적으로 뭘 하는 건가요?

기존 방식

📖 문법을 배우고
한 줄씩 직접 작성

내가 규칙을 외우고, 내가 손으로 만듭니다.

바이브 코딩

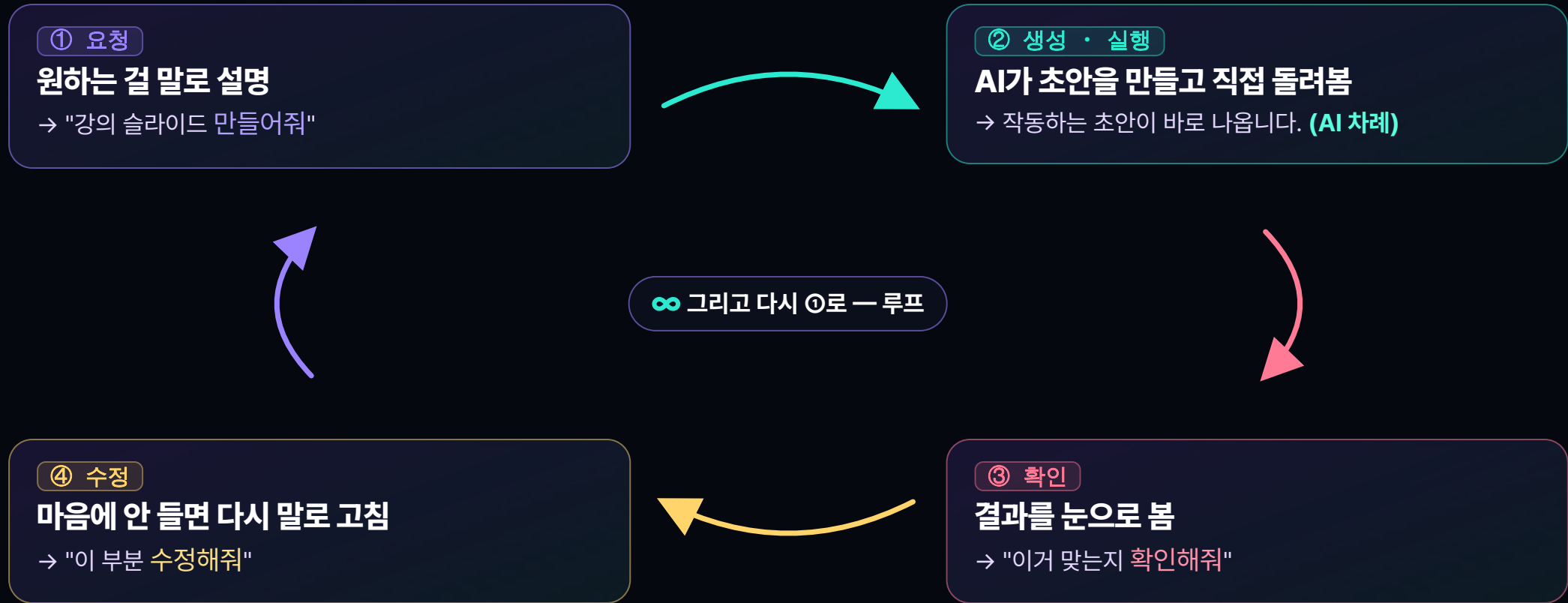
💬 원하는 걸 말로 설명하고
AI가 만들고 실행

내가 방향을 말하고, AI가 초안을 만들어 돌려봅니다.

단, AI는 완벽한 정답이 아니라 "가장 그럴듯한 초안"을 줍니다.
그래서 한 번에 끝나지 않고, 방향을 주고 → 결과를 보고 → 다시 다듬는 주고받음이 필요해요.

그리고 이 주고받음은, 늘 똑같은 리듬으로 돕니다 ↓

그 리듬이, 바로 이 루프입니다.



중요한 건 "코드를 아는 것"이 아니라, "원하는 걸 표현하고 결과를 판단하는 것"입니다.

* "vibe coding"이라는 이름은 2025년 초, AI 연구자 Andrej Karpathy가 붙였습니다.

백문이 불여일견 — 루프가 결과물을 키우는 장면.

방금 그 루프를 그대로 돌려보죠. 오른쪽 위젯은 지금 이 화면에서 실제로 작동 중입니다.

CLAUDE

기본 타이머를 만들었어요. 바로 실행해 보세요 →

YOU

회의 때 쓸 타이머 위젯 하나 만들어줘.

YOU

너무 맛있는데. 다크 테마에 카드 느낌으로.

CLAUDE

다크 카드 + 모노 폰트로 바꿨습니다 →

YOU

좋아. 남은 시간이 한눈에 보이게 — 링 같은 걸로.

CLAUDE

진행률 링을 얹었어요. 시간이 줄면 링도 줄어듭니다 →

YOU

마지막으로 우리 발표 컬러(보라·민트)로 맞춰줘.

CLAUDE

완성입니다. 코드는 한 줄도 직접 안 쓰셨어요 →

meeting-timer

v1 · 기본

05:00

TEAM SYNC

① 요청 → ② 생성 → ③ 확인 → ④ 수정 — 방금 그 네 박자가 세 바퀴 돈 겁니다.

그러면 AI로 생산성을 올린다는 건 대체 무슨 뜻일까요?

흔한 답

⚙️ 자동화 — 사람 일을 대신하기

물론 훌륭합니다. 하지만 이건 절반의 정답입니다.

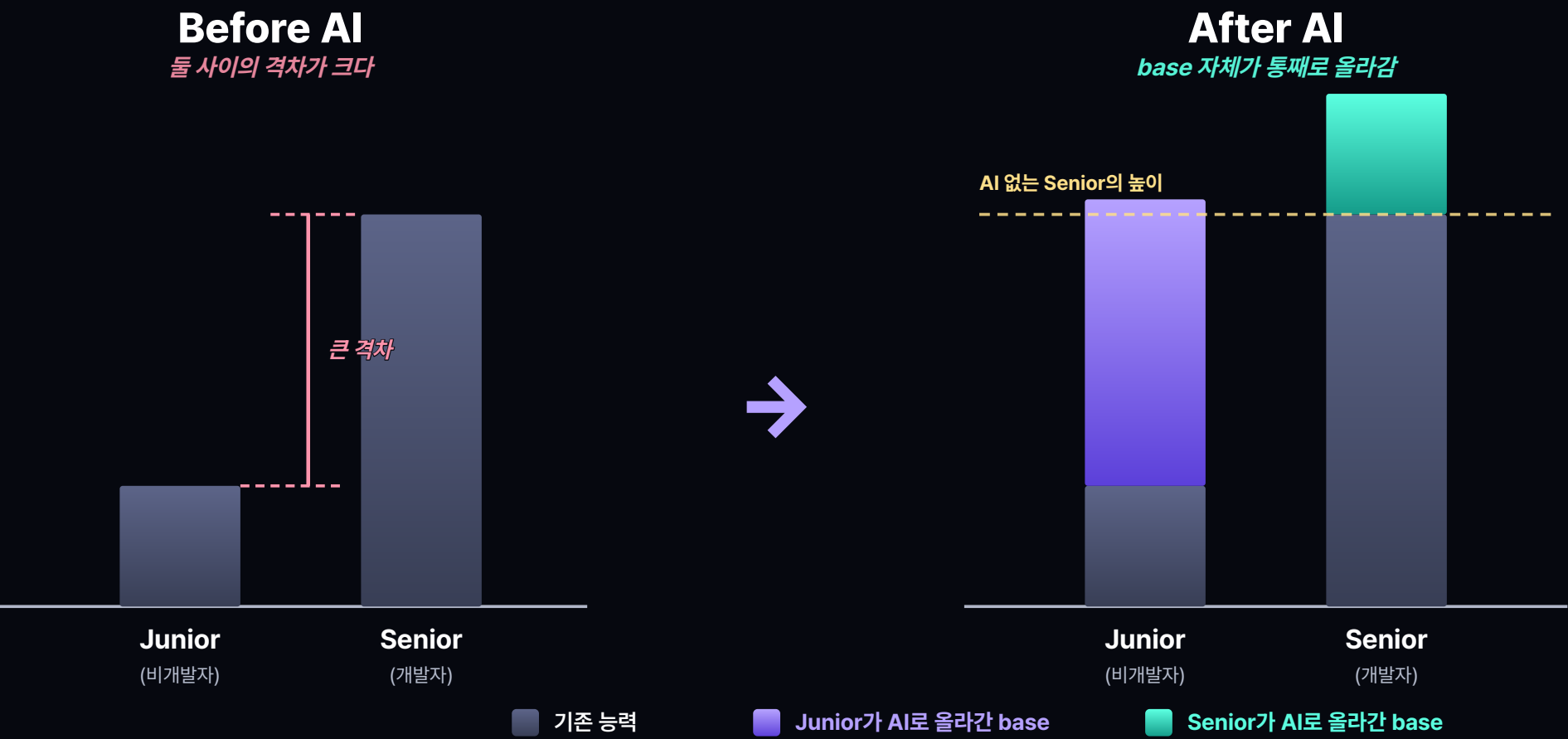
진짜 답

🚀 인간 능력의 base를 끌어올리기

AI는 사람과 결합해서 그 사람의 능력 자체를
비약적으로 끌어올립니다.

이런 그림이에요.

주니어 ↔ 시니어의 이야기이자, 동시에 비개발자 ↔ 개발자의 이야기입니다.



AI를 쓰는 주니어는 AI를 안 쓰는 시니어를 넘어설 수 있습니다.
그럼에도 격차는 남습니다 — 다만 그 축이 연차·직군에서 "AI를 쓰느냐"로 바뀝니다.

"base가 올라간다"는 건, 이런 순간들입니다.

숫자에 약한 마케터

"손익계산서만 보면 머리가 하얘져요."

→ 재무제표를 직접 읽으며 회의에서 질문합니다. AI가 매 순간 옆에서 해설해 주니까요.

처음 예산안을 짜는 신입

"예산안은 한 번도 안 만들어봤는데..."

→ "보통 이렇게 짤다"는 역사상의 표준안을 초안으로 받고, 다듬는 데만 집중합니다.

낯선 산업으로 이직한 사람

"이 업계는 용어부터 낯설어요."

→ "이 도메인은 이런 흐름으로 굴러간다"는 패턴을 몇 년이 아니라 첫 주에 흡수합니다.

새 프로젝트를 여는 팀

"비슷한 걸 해본 사람이 팀에 없어요."

→ 유사 프로젝트들의 성공·실패 지점을 키포프 자료로 바로 가져옵니다.

공통점 — 코딩이 아닙니다. "그 일을 이미 해본 수만 명"이 옆자리에 앉는 경험입니다.

...부작용이 하나 있다면, 이제 "저는 그거 할 줄 몰라서요..."라는 핑계가 안 통하는 시대가 됐다는 것 😊

좀 더 작게 — 어느 하루로 내려가 볼까요?

09:12

받은메일함부터 꼭 차 있음

"이 답장, 남들은 보통 어떻게 하지?"

→ 톤까지 맞춘 초안 3개 중에서 **고르기만** 합니다.

11:40

처음 보는 계약서 검토

"이런 조항, 남들은 보통 어떻게 하지?"

→ **위험한 문장부터** 형광펜을 그어 줍니다.

14:30

숫자만 빼 곡한 스프레드시트

"이 데이터, 남들은 보통 어떻게 하지?"

→ 이상한 값과 핵심만 **세 줄로** 짚어 줍니다.

16:50

처음 켜 보는 낯선 툴

"이거 시작, 남들은 보통 어떻게 하지?"

→ 첫 화면부터 **손을 잡고** 안내합니다.

네 번 다 하는 일은 다른데, 뒷말은 **토씨까지 똑같았습니다** — "보통 어떻게 하지?"

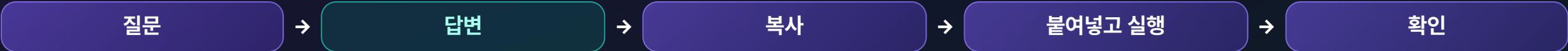
AI는 그때마다 '지금까지 가장 승률 높았던 답'을 꺼내 옆자리에 놓아 줍니다.

그리고 이 한 문장 — 그냥 **첫뱃 창에 쳐 넣는 것도**, 이미 그 문턱입니다.

그럼 챗봇 창에 물어보는 것도 바이브 코딩인가요?

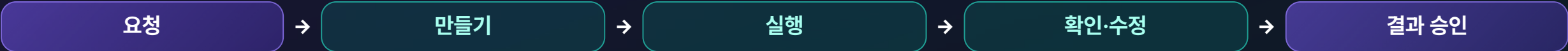
네, 이미 시작입니다. 다만 누가 손발이 되느냐가 다릅니다 — 보라 = 나 민트 = AI

대화형 — ChatGPT, Claude.ai (웹)



AI는 조언만 — 손발은 전부 나. 왕복이 잦을수록 여기서 시간이 썩니다.

실행형 — Claude Code, Cursor, Codex



나는 방향과 판단만 — 중간 루프가 통째로 AI에게 넘어갑니다.

한 번 쓰고 끝이면 챗봇으로 충분, 반복해서 만들고 돌려야 한다면 실행형이 유리 — 그래서 "어떤 도구를 배워야 하나"는 질문이 남죠.

자, 그러면 본론이에요.

많은 분들이 묻습니다. "Claude Code? Cursor? Codex? 뭐 배울까요?"

요즘은 여기에 "MCP는요? Skill은요? 어떤 harness 깔면 돼요?"까지 붙습니다.

지난 5년, 코딩 AI 도구의 변천사.

2021-2022

IDE Autocomplete

Copilot, Tabnine. 코드를 치다 보면 "다음 한 줄을 채워주는" 시대.

2022-2023

Chatbot에 물어보고 복붙

ChatGPT 등장. 창에 질문 → 답을 복사 → 코드에 붙여넣기.

2023-2024

AI-native Desktop App

Cursor, Windsurf. IDE 자체가 LLM을 중심으로 재설계.

2024-2025

Terminal-based Agent

Claude Code, Codex CLI. 터미널(명령창)에서 파일을 직접 만지고 명령까지 실행하는 에이전트.

2025-현재

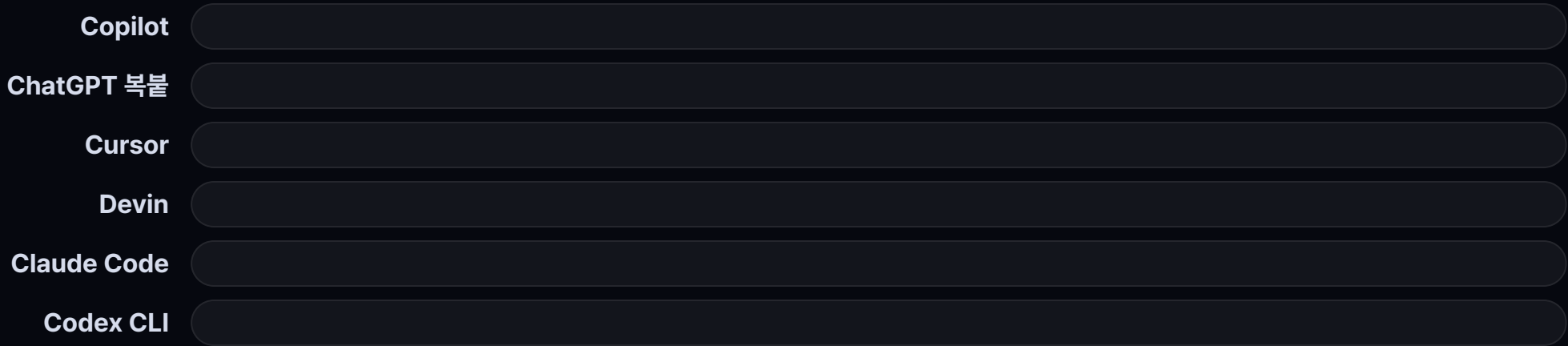
Agent + 확장 생태계

MCP · Skill · Subagent · Harness. 에이전트 위에 새로운 층이 계속 얹히는 중.

왕관은 계속 바뀌어 왔습니다.

"어떤 AI 코딩 도구가 최고냐" — 지난 5년의 레이스를 직접 돌려보죠.

2021



그중 몇몇은 벌써 이름조차 뜸해졌고 — 3개월 후는 아무도 모릅니다. 그게 함정입니다.

※ 막대는 시기별 개발자 커뮤니티의 체감 인기를 재구성한 개념도입니다 — 정확한 점유율이 아닙니다.

매일 듣는 질문

"그래서, 뭐 하나만 깔면 되는 거죠?"

Q

"어떤 **harness** 깔아요?"

Claude Code? Codex? 하나만 정해주세요.

Q

"**MCP**는 뭐부터 깔아요?"

무슨 서버를 붙여야 '완성'인가요?

Q

"**Skill**은 어디서 받아요?"

그거 설치하면 다 되는 거 맞죠?

다들 "하나만 세팅하면 다 되는 그거"를 찾습니다. — 그런 건, 없어요.

이건 전부 '표면'이에요.

도구·모델·MCP·skill·harness — 매 분기 새로 얹히고, 또 바뀝니다.

▲ 표면 - 계속 churn

harness	Claude Code · Codex CLI ...	3개월마다 바뀔
skill · MCP	확장 플러그인	계속 새로 생김
model	GPT ↔ Claude ↔ Gemini	순위 뒤집힘
tool · IDE	Cursor · Windsurf ...	표준이 바뀔
AI의 정체 + 선택하는 사고방식		안 바뀔

"어떤 harness 깔지"가 아니라, "무엇을 안 바뀌도 되는지"를 배우세요.

그래서 저는 이렇게 생각합니다 —

AI 도구의 사용법을

**깊게 배우는 건
의미가 없습니다.**

왜냐면...

① 표면은 전부 빨리 바뀐다

도구·모델·MCP·skill·harness — 그 위에 새 층이 계속 생깁니다. Cursor 단축키를 외우고 나면, 6개월 뒤엔 다른 게 표준이 되죠.

② 새 층이 생길 때마다 같은 착각이 반복된다

"이번엔 이거 하나만 세팅하면 돼." — MCP도, skill도, harness도 그랬어요. 그리고 다음 달, 또 새로운 '이거 하나'가 등장합니다.

③ 그러면 무엇이 변하지 않을까?

→ "AI라는 것의 본질"과, "AI를 다루는 나의 사고방식"

그래서 진짜 길러야 할 능력은요.

Skill 1

AI의 본질을 이해한다

"AI는 역사 속에서 가장 승률 높은 답을 추천하는 기계다."

이것만 진짜로 이해하면, 어떤 도구가 와도 적응할 수 있습니다.

Skill 2

모든 일에서 AI에게 조언을 받고, 선택하고, 다시 묻는 사이클을 연습한다

구현, 수정, 설계, 업무 — 어디서든.

이건 근육입니다. 도구가 바뀌어도 그대로 갑니다.

그리고 한 가지 더 —
그 선택은 **주체적**일수록 좋습니다.



AI의 강점

"**단기적**으로 가장 그럴듯한 답"을 빠르게 찾아준다.



AI의 약점

"**장기적** 옳음"엔 약합니다.

단기 그럴듯함에 과적합하는 경향이 있어요.

→ 그래서 마지막 결정은 **사람**이 해야 합니다.

AI가 추천한 답을 그대로 따르지 말고, "이게 장기적으로도 옳은가"를 한 번 더 묻는 습관.

AI를 쓸수록 더 똑똑해지는 사람.

AI는 사람을 자동으로 똑똑하게 만들지 않아요. 어떤 태도로 쓰느냐가 중요합니다.

전문성 · 맥락화

AI 답을 판별한다

맞는지 틀린지는 결국 내가 가려냅니다. 도메인 지식이 있어야 환각·누락·과장이 보여요.

메타인지

모름을 안다

뭘 모르는지 알아야 더 정확히 묻고 검토합니다. 진짜 학습은 답을 내 언어로 다시 쓸 때 일어나요.

AI를 안 쓰는 시간

혼자 생각한다

잘 쓰는 사람일수록 AI 없이 사색하는 시간을 지킵니다. 독서·경험이 사고 근육을 만들어요.

AI는 대체물이 아니라 **증폭기**다 — 무엇을 위임할지보다 **무엇을 끝까지 내 사고로 남길지**가 더 중요합니다.

참고문헌 · 『AI, 신의 탄생 인간의 종말』, 『듀얼 브레인』

RECAP

여기까지의 한 페이지 요약.

1. AI는 "역사상 가장 승률 높은 다음 답"을 찾아주는 확률 계산기다.
2. 그래서 LLM도 본질은 "다음 단어 예측기"다. 하지만 강력하다.
3. AI 생산성의 진짜 의미는 "인간 능력의 base를 끌어올리는 것".
4. 도구의 사용법이 아니라 AI의 본질과 선택 근육을 길러라.
5. 그리고 그 선택은 주체적일수록 좋다 — AI는 단기 최적화에 치우치니까.

이 다섯 줄을 들고 — 처음 질문으로 돌아가서, 실전으로 넘어갑니다.

그래서, 다시 처음 질문으로 —

바이브 코딩이란 무엇인가?

어떤 도구·harness를 깔았느냐가 아니라,

AI라는 지능형 도서관과 끊임없이 대화하면서
매 순간 가장 좋은 선택을 골라나가는 작업 방식

그게 vibe coding입니다.

LAST EXAMPLE

마지막에, 가장 간단한 예시 하나.

어떤 도구를 쓰든 — AI를 가장 강력하게 끌어내려면, 어떻게 해야 할까요?

흔한 방법

바로 부탁한다

"이거 만들어줘" 한 방.

물론 이것도 좋습니다. 하지만 한계가 있어요.

더 강력한 방법

가이드라인을 같이 잡아간다

대화하며, 선택지를 비교하고, 한 단계씩 결정해 나가기.

이 방식엔 세 가지가 한 번에 일어납니다 ↓

① 성공 확률이 가장 높은 선택

② 그 선택을 하면서 학습이 일어남

③ AI의 단기 뷰 편향을 막을 수 있음

예를 들어, 이런 흐름이에요.

YOU

100명 대상으로 1시간짜리 **바이브 코딩 개론** 슬라이드를 만들고 싶어. 어떻게 시작할까?

CLAUDE / CODEX

가장 무난한 건 **PowerPoint**입니다. 그 외 Keynote, HTML 슬라이드(`reveal.js`)도 있어요 — 디자인 자유도가 얼마나 필요한가요?

YOU

애니메이션이 좀 더 돋보이고, 디자인도 **현대적**이었으면 좋겠어.

CLAUDE / CODEX

그럼 **HTML 기반**이 가장 자유롭습니다. `reveal.js` + 커스텀 CSS로 충분히 됩니다.

YOU

청중이 **실시간**으로 참여하는 요소도 넣으려면?

CLAUDE / CODEX

그건 입력을 모을 **서버**가 필요해요. **AWS EC2 + React** 웹앱화를 추천 — 실시간 입력까지 됩니다.

YOU

만들었더니 **4명만** 써도 느려져. 원인이 뭘까?

CLAUDE / CODEX

접속자마다 서버가 같은 데이터를 **매번 다시 만들어 보내는** 게 주범 같아요. ① **캐시**, ② **I/O 큐 관리** 두 방향을 보세요.

YOU

그중에서도 **실시간성**이 가장 중요하다면?

CLAUDE / CODEX

Redis 도입을 검토하세요. 캐시 + Pub/Sub으로 동시성·지연을 한 번에 잡을 수 있습니다.



그리고 이 대화의 결과물이 — 지금 보고 계신 이 슬라이드입니다.

이 짧은 대화에서, 세 가지가 한꺼번에 일어났습니다.

①

성공률이 가장 높은 선택

"PPT vs HTML vs React" 같은 갈림길에서,
매번 조건에 맞는 답을 추천받음.

②

학습이 자연스럽게 따라옴

Redis, Pub/Sub, 큐 — 몰랐던 개념을 필요한
순간에 정확히 배움.

③

단기 뷰 편향을 막음

"그냥 만들어줘"였다면 첫 답인 PPT로 끝났을
것. 질문을 거듭하니 장기적으로 옳은 길이 보
임.

"답을 받는 것"이 아니라 "같이 길을 잡아가는 것" —
그게 바로 *vibe coding*입니다.

그럼 Claude Code 어떻게 쓰나요?

설치부터 실전까지 — 기본기 · 명령어 · Plugin · MCP · Agent Skills

아까 "사용법을 깊게 배우지 말라"고 했죠. 맞습니다 —
여기서는 루프를 돌리는 데 필요한 최소한만 다룹니다.

설치·실행

기본 조작

슬래시 명령어

Plugins

MCP

Agent Skills

✨ 뒤에 다시 펼쳐진 이 별밭 — 첫 화면의 미니게임까지, 전부 이 자료를 만들며 바이트 코딩으로 만든 결과물입니다.

쓰기 전에 — 안에서 무슨 일이 벌어지나.

프롬프트 하나를 주면, Claude Code는 **작업이 끝날 때까지** 이 루프를 스스로 돕니다.

에이전틱 루프

① 맥락 수집
파일 검색·읽기로 코드 이해



② 실행
편집·명령·테스트 실행



③ 검증
결과 확인 · 틀리면 교정

↻ 세 단계는 뒤섞이고 반복됩니다 — 질문 하나엔 ①만, 버그 수정엔 ①②③을 수십 번.

하네스 = 모델 + 도구

모델이 추론하고 **도구**가 행동합니다. Claude Code는 이 둘을 감싸 **언어모델을 유능한 에이전트로** 바꾸는 껍데기(harness)일 뿐입니다.

당신도 루프의 일부

자율적으로 돌지만 **언제든 Esc로 끼어들어** 방향을 바꾸거나 맥락을 더 줄 수 있습니다.

그래서 **인라인 자동완성**과 다릅니다 — 프로젝트 전체를 본다.

현재 파일만 보는 어시스턴트와 달리, 여러 파일을 넘나들며 조율된 수정을 합니다.

볼 수 있는 것 · 접근 범위

- 📁 프로젝트 파일 — 폴더·하위폴더 전체 (그 폴더가 곧 작업 범위)
- 💻 터미널 — 당신이 칠 수 있는 모든 명령 (빌드·git·패키지...)
- 🔗 git 상태 — 현재 브랜치·변경분·최근 커밋
- 📄 CLAUDE.md · 자동 메모리 — 세션마다 기억할 규칙·패턴
- 🛠️ 확장 — MCP · Skills · Subagents

할 수 있는 것 · 도구 5범주

- 파일 조작 — 읽기·편집·생성·정리
- 검색 — 패턴으로 파일 찾기·정규식 내용 검색
- 실행 — 셸 명령·서버·테스트·git
- 웹 — 검색·문서 조회·에러 메시지 찾기
- 코드 인텔리전스 — 편집 후 타입 오류·정의 이동 (플러그인)

도구를 쓸 때마다 나온 결과가 **다음 판단**으로 되먹임됩니다 — 그게 방금 본 에이전틱 루프.

시작하기

설치는 한 줄 — 접속 설정은 환경 따라, 실행은 폴더 안에서.

my-project - terminal

```
$ npm install -g @anthropic-ai/claude-code
added 1 package in 4s
$ aws sso login # 사내 Bedrock이라면 - 환경변수 설정 후
$ claude
```

```
* Welcome to Claude Code!
/help for help · cwd: D:\work\my-project
```

> 이 프로젝트 구조부터 설명해줘



설치 자체는 이 한 줄

Node.js만 있으면 어떤 OS든 여기까진 동일합니다.



접속 설정은 두 갈래

개인 — 첫 실행 때 Claude.ai 유료 플랜(또는 API 키)으로 로그인 한 번.
사내 AWS Bedrock — 환경변수 설정에, 세션이 만료될 때마다 aws sso login까지. 한 줄로는 안 끝납니다 — 사내 가이드를 따라 주세요.



시작 위치가 중요

반드시 작업할 프로젝트 폴더 안에서 실행 — 그 폴더가 곧 작업 범위입니다.



그 다음은 그냥 말하기

명령어를 몰라도 됩니다. 원하는 걸 한국어로 적으면 그게 시작입니다.

슬래시 명령어보다 더 자주 쓰는 것들.

my-project - claude

> 이 화면 디자인을 참고해서 @

@src/components/Header.tsx

@src/styles/theme.css

@README.md

[Image #1] 스크린샷 붙여넣음 (Ctrl+V)

● 작업 중... (Esc를 누르면 즉시 중단)



@ 파일 참조

@만 치면 파일·폴더가 검색됩니다. "그 파일 어디 있더라" 설명할 필요가 없어집니다.



이미지 붙여넣기

화면을 캡처해 **드래그**하거나 **Ctrl+V**만 해도 바로 시각적 맥락으로 활용됩니다.

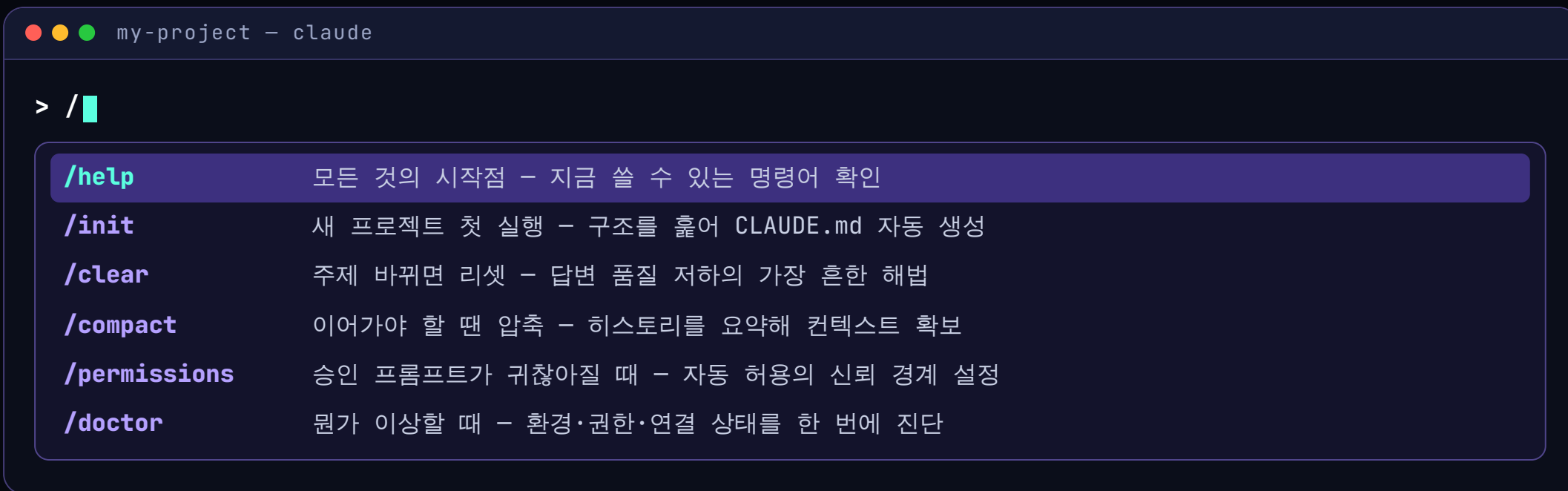


Esc — 엉뚱하면 바로 멈추기

원하는 방향이 아니면 끝까지 기다리지 말고 **즉시 중단** 후 다시 지시하세요.

가장 먼저, 이 6개부터 손에 익히세요.

외울 필요 없습니다 — /만 입력하면 이 메뉴가 뜹니다.



위에서 아래 순서대로 — 처음 며칠 안에 저절로 손에 붙는 **우선순위**입니다.

권한 모드와 Plan Mode — 안심하고 맡기는 법.

my-project - claude

> 빌드 스크립트를 정리해줘

● Bash(npm run build)

이 명령을 실행할까요?

- > 1. Yes 이번만 허용
- 2. Yes, always 앞으로 자동 허용
- 3. No 다른 방법을 지시 (Esc)

|| plan mode on — Shift+Tab으로 모드 전환



기본 — 매번 승인 요청

파일 수정·명령 실행마다 이 화면으로 확인. 처음엔 이 모드가 안전합니다.



자동 승인 — 파일 수정만 자동

신뢰가 쌓이면 전환. /permissions로 허용 범위를 세밀하게 조정.



Plan Mode — 계획부터

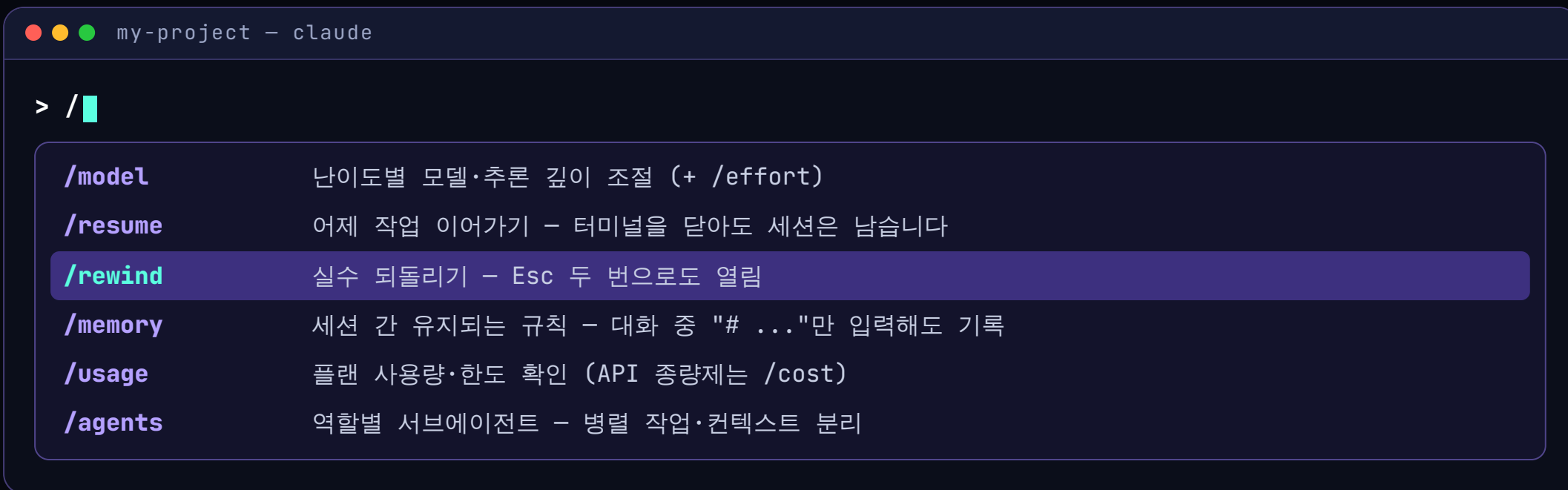
바로 고치지 않고 계획을 먼저 승인받은 뒤 실행. 큰 작업 전엔 필수.



Bypass — 전부 자동

모든 승인 생략. 격리된 샌드박스가 아니면 권장하지 않습니다.

손에 익으면, 이 6개도 챙기세요.



→ 사내 도입 시엔 Pro·Max·Team 중 어떤 플랜인지, 사용량이 개인별인지 팀 단위인지 관리자에게 확인해두면 좋습니다.

방금 그 /model — 4개 중 언제 뭘 쓸까요?

외울 필요 없습니다. 대부분 Sonnet이면 되고, 필요할 때만 갑니다.

가벼운 보조

Haiku

싸고 빠른 인턴. 메인 구현엔 안 씁니다.

긴 로그 훑기 · 여러 파일 요약 · 분류·추출 ·
diff 의도 정리

기본값 · 주력

Sonnet

일상 코딩의 기본 루프. 속도·품질·비용 밸런스 최고.

기능 구현 · 버그 수정 · 테스트 작성 · 일반 리팩토링

설계·리뷰

Opus

비싼 시니어. 판단을 시키는 모델입니다.

아키텍처 결정 · 복잡한 원인 분석 · 위험 변경·PR 리뷰

장기·대형

Fable

머칠짜리 자율 에이전트. 매일 켜는 모델이 아닙니다.

대규모 마이그레이션 · 설계→구현→테스트→문서 일괄

Fable은 최신 버전에서만 보이고, 응답이 느리며 사용량을 많이 씁니다 — 정말 큰 덩어리를 통째로 맡길 때만.

실전 운영법 — 거의 Sonnet, 가끔 갈아탑니다.

골든 루프

Opus로 설계·분해·리스크



Sonnet으로 구현



Opus로 최종 리뷰

비싼 시니어(Opus)에게 판단을 맡기고, 손발(Sonnet)에게 구현을 시키는 흐름.

막혔을 때

Opus에게: "지금까지 시도·실패 로그·코드 구조를 보고 원인 후보를 좁혀줘."

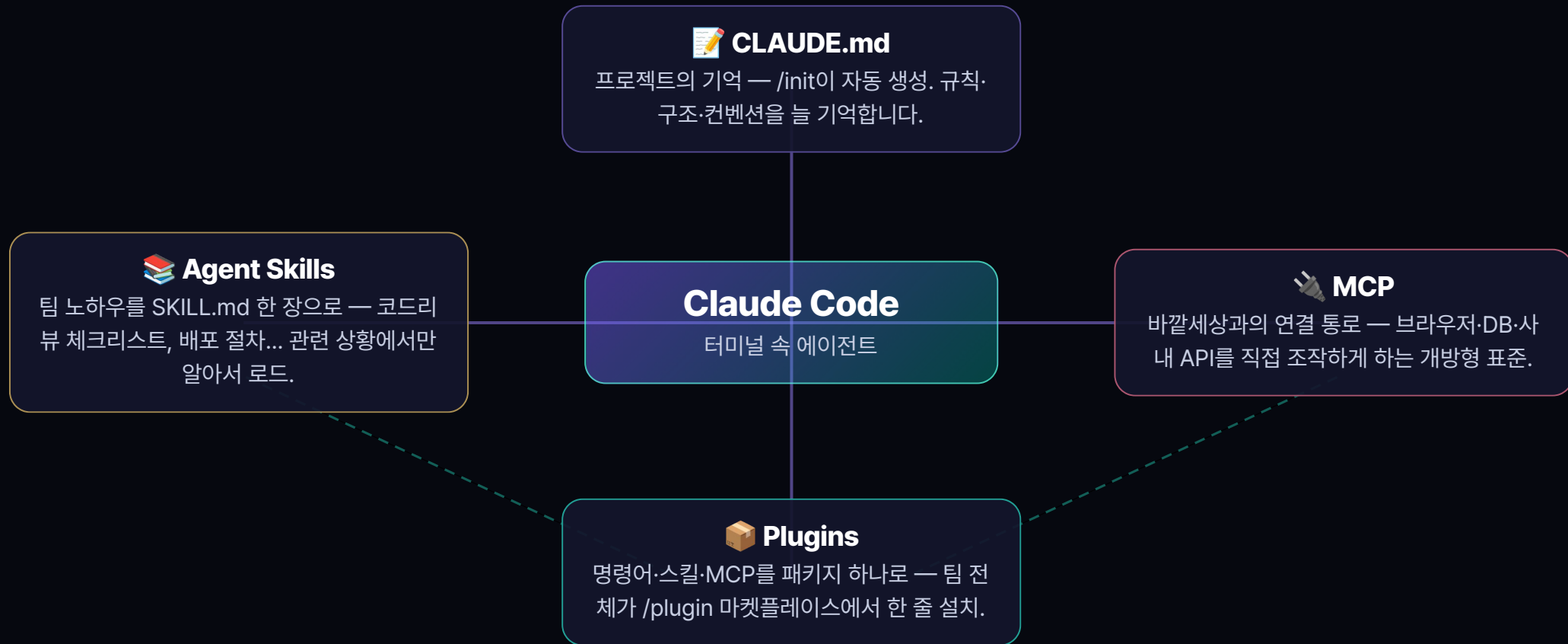
정말 큰 작업

Fable엔 "어떻게"가 아니라 목표·제약·완료 조건을 주고 통째로 위임.

대략적인 비중

Sonnet 80% · Opus 15% · Haiku 4% · Fable 1% — 대형 마이그레이션이 잦다면 Fable 비중을 올리세요.

확장은 필요한 만큼만 — 생태계 지도 한 장.



※ 사내망에선 GitHub·Slack 같은 사외 SaaS 연동 MCP가 막혀 있는 경우가 많습니다 — 사내에서 되는 것 위주로.
이 세미나 자료의 화면 검증도 실제로 브라우저 MCP(Playwright)로 진행했습니다.

Skill·Plugin, 어디서 구하고 뭘 조심해야 할까요?

설치처

skills.sh 마켓플레이스

인기 스킬을 검색해서 `npx skills add` 한 줄로 설치.

왜 조심해야 하나

SKILL.md = 자연어 지시문

출처 불명 스킬은 **프롬프트 인젝션**의 통로가 될 수 있습니다.

설치 전 체크리스트

popular = safe는 아니지만, 최소한의 필터는 된다

인기도(스타·설치 수)와 검증된 저자·조직 위주로 고르고 — 설치 전엔 평문인 SKILL.md 원문을 직접 읽어볼 것.

"반드시", "무조건 따라야 한다"처럼 **강압적인 어투**가 보이면 인젝션을 의심하세요.

→ 이 세미나를 준비하는 동안에도, 도구 결과 속에 "반드시 따라야 한다"는 낯선 지시문이 섞여 들어온 적이 있습니다 — 전형적인 인젝션 패턴이라 무시했습니다.

튜닝 전에

튜닝의 끝은 순정 — 확장은 나중에.

MCP·hook·skill·subagent를 붙이기 전에 — 기본 상태의 Claude Code가 어디서 잘하고 어디서 틀리는지부터 몸으로 익히세요.

요행을 기대하지 말 것

"좋은 MCP만 붙이면 알아서 잘하겠지."

"고수의 CLAUDE.md를 복사하면 되겠지."

"subagent 여러 개면 자동으로 굴러가겠지."

"hook을 걸면 실수는 다 막아주겠지."

확장은 기본기를 대체하지 않습니다. 기본기를 가진 사람이 쓸 때만 효과가 납니다.

좋은 순서

순정으로 써본다

→

불편·실패를 기록

→

그때 설정을 추가

남의 세팅을 복사하는 게 아니라 — 내 업무·코드베이스·보안 환경·팀 방식에 맞게 에이전트 환경을 키워가는 것.

순정을 제대로 다룰 줄 아는 사람이 — 나중에 가장 좋은 튜닝도 합니다.

순정 다음, 확장한다면

순정이 손에 익었다면 — 이 3가지부터.

전부 강력하지만 공짜가 아닙니다 — 컨텍스트·토큰을 더 씁니다. 반복되는 병목이 확인된 뒤에 하나씩.

범용 워크플로우

Superpowers

브레인스토밍, 체계적 디버깅 등 검증된 작업 절차를 모아둔 스킬 모음집.

🗑️ 절차가 다단계·서브에이전트를 부릅니다 — 작업당 토큰↑.

스킬 제작

Skill Creator

내 팀만의 노하우를 SKILL.md로 뽑아내는 메타 스킬. 스킬 만드는 스킬.

● 만들 때만 잠깐 — 상시 비용은 거의 없음.

브라우저 자동화

Playwright MCP

웹 화면을 직접 보고 조작하며 검증. 프론트엔드 작업의 강력한 도구.

🗑️ 매 조작마다 스냅샷/스크린샷이 컨텍스트로 — 검증 루프에서 토큰↑↑.

저예산이라면 — 예: 하루 \$10 수준의 종량 API

순정 + /model(Sonnet-Haiku)만으로도 대부분 됩니다. Playwright는 화면 검증이 꼭 필요할 때만 켜고 끝나면 끄세요 — 정액 플랜(Max)이 아니면 확장이 쓰는 토큰은 그대로 청구됩니다.

확장을 붙여도, 결국 "생각하고 → 만들고 → 눈으로 확인" 루프가 핵심입니다.

AI 시대에, '사용법'을 가르치는 게 맞을까?

이 가이드를 준비하며 든 묘한 불편함 하나.

AI 시대라면서 — 우리는 여전히 설치·명령어·모델 선택·권한 설정 **사용법**을 하나하나 설명하는 세미나를 열고 있습니다.

정말 중요한 건 "**Claude Code 사용법**"을 아는 것일까,
아니면 "**모르는 도구를 AI와 함께 익혀가는 능력**"을 갖추는 것일까?

저는 후자가 훨씬 중요하다고 생각합니다.

관점의 전환

도구 교육의 중심이 달라져야 합니다.

예전 도구 교육

"정답 절차"를 알려주기

메뉴가 어디 있는지 · 명령어가 무엇인지 · 정해진 순서가 무엇인지. **외우면** 끝나는 지식.

AI 에이전트 도구 교육

"함께 일하는 법"을 익히기

원하는 작업을 어떻게 설명할까 · 실패하면 어떻게 다시 대화할까 · 시행착오를 규칙 · skill · hook · MCP로 어떻게 축적할까.

Anthropic의 실제 사용 분석: 사람은 주로 **"무엇을 할지"**(계획), Claude는 **"어떻게 할지"**(실행)를 맡았고 — 사용자의 **도메인 전문성이 높을수록** 한 번의 지시로 더 많은 일을, 더 높은 성공률로 해냈습니다.

→ AI 시대에도 전문성은 사라지지 않습니다. 다른 방식으로 증폭될 뿐.

사용자 → 설계자

좋은 질문을 던지는 순간, 설계자가 됩니다.

my-project - claude

- > 이 프로젝트 구조를 읽고 설명해줘.
- > 내가 뭘 먼저 해야 할지 작업 계획을 세워줘.
- > 이 에러 로그를 보고 원인 후보를 좁혀줘.
- > 다음부터 더 잘하려면 CLAUDE.md에 뭘 남기면 좋을까?
- > 이 명령 매번 치기 귀찮은데, custom command로 만들 수 있을까?
- > 팀 코드 스타일·테스트 규칙을 에이전트가 기억하게 하려면? ■



실력이 이동한다

"AI에게 한 번 물어보는 능력"이 아니라 "AI가 계속 잘 일할 환경을 만드는 능력"으로.



반복은 승격시켜라

같은 설명을 반복 중이라면 — 프롬프트가 아니라 CLAUDE.md·command·hook·skill·MCP로 남길 순간입니다.



가이드는 '출발선'일 뿐

보안·권한·사내코드 탓에 기본 가이드는 필요합니다. 목적은 "이대로 따라 하세요"가 아니라 "안전하게 출발하는 공통 기반".

그래서, 이 세미나를 이렇게 정의합니다.

이 세미나는 Claude Code의 **모든 사용법**을 알려주는 시간이 아닙니다.
함께 스스로 학습하고, 실패를 개선하고, **자신의 업무 환경에 맞는 에이전트 작업 방식**을 만들어가기 위한 **출발점**입니다.

1. 일단 써보자.
2. 막히면 Claude Code에게 물어보자.
3. 잘 안 되면 왜 안 되는지 함께 분석하자.
4. 반복되는 문제는 규칙과 도구로 남기자.
5. 각자의 업무에 맞는 **더 좋은 에이전트 환경을 스스로** 만들자.

그 과정 자체가 — **AI 시대의 실력**입니다.

바이브 코딩에서 에이전틱 엔지니어링으로

그리고 — 비개발자에게 여전히 중요한 바이브 코딩

같은 AI를, 개발자와 비개발자가 다른 방식으로 극대화하는 이야기.

바이브 코딩은 문을 열었지만, 벽도 분명했습니다.

열어준 것

자연어 프롬프트만으로 빠르게 코드를 만들며 진입장벽을 크게 낮췄다.
아이디어가 떠오르면 바로 시제품으로. 비개발자도 소프트웨어 제작에 참여
할 수 있게 됐다.

부딪힌 벽

환각으로 인한 오류 · 복잡한 디버깅 · 보안 취약점 수정에 오히려 더 많은 시
간.
구조 없는 즉흥 개발은 유지보수 비용을 키우고 — 테스트·설계가 빠진 코드는
빠르게 기술 부채가 됐다.

가능성은 열었지만 — 지속 가능한 개발을 위해선 더 체계적인 접근이 필요했습니다.

이제 AI는 도구가 아니라, 엔지니어링 파트너.

계획



구현



검증

코드를 바로 쓰기보다 — 먼저 아키텍처·작업 계획을 세우고, 구현 뒤엔 테스트·검증을 자동화한다.



계획 우선

즉흥 생성 대신 설계·계획이 먼저.



병렬 에이전트

프론트·백엔드·문서·테스트를 나눠서 동시에.



MCP 도구 표준

파일·DB·외부 API 접근을 안전하게 통제.

핵심은 — AI 활용을 "재미있는 실험"에서 "신뢰 가능한 개발 프로세스"로 끌어올리는 것.

작성자에서, 시스템 오케스트레이터로.

코드를 많이 타이핑하는 사람



에이전트가 안전하게 일할 구조를 만드는 사람

범위

수정 범위를 제한

에이전트가 건드릴 수 있는 곳을 명확히 가둔다.

의도

규칙·의도를 전달

코딩 규칙과 아키텍처 의도를 명확히 전한다.

검토

결과물을 리뷰

품질·보안·운영 관점에서 프로덕션 수준으로.

직접 모든 코드를 쓰기보다 — 구조를 만들고, 검토하고, 승격시키는 사람.

비개발자에게 바이트 코딩은 여전히 강력한 도구.

현업 문제를 가장 잘 아는 사람은 개발자가 아니라 — 그 업무를 직접 하는 도메인 전문가.

이들이 자연어로 요청해 작은 자동화 도구·대시보드·챗봇·업무 앱을 직접 만들면 — 문제 해결 속도가 크게 빨라진다.

바이트 코딩은 개발자를 **대체하는 수단**이 아니라 —

도메인 문제 해결력을 증폭하는 도구.

중요한 건 코드 실력보다 — 문제를 정확히 정의하는 능력.

비개발자의 생산성은 도메인 지식 × AI 활용.



직접 움직여보세요 — 한쪽만 10이어도 크지 않습니다. 덧셈이 아니라 곱셈이니깐요.

정식 개발 과제가 되기엔 작지만, 현업에선 매일 시간을 잡아먹는 문제들 —

📧 메일·엑셀 반복 취합

📊 CSV 정리 · 시각화

🔍 팀 문서 기반 검색

💬 FAQ 챗봇

📅 작은 계산기 · 시뮬레이터

📈 업무용 대시보드

도메인 지식이 **깊을수록** — AI로 만들 수 있는 것이 **폭발적으로** 커집니다.

선택이 아니라 — 함께 갈 때 가장 강합니다.

비개발자 · 바이트 코딩

문제를 빠르게 발견하고 푼다

현업 문제를 발견하고 · 빠르게 시제품을 만들고 · 작은 업무 개선을 직접 실험한다.

개발자 · 에이전틱 엔지니어링

안정적인 시스템으로 완성한다

그 아이디어를 더 안전하고 · 확장 가능하고 · 운영 가능한 시스템으로 승격시킨다.

개발자 협업이 필요해지는 순간 —

사용자 급증

민감정보 · 보안

시스템 연동 · 장기 운영

바이브 코딩은 끝난 트렌드가 아닙니다.

다만 모든 문제를 바이브 코딩으로 끝내려는 접근이 위험할 뿐.

비개발자에게

바이브 코딩은 도메인 문제 해결력을 극대화하는 도구.

개발자에게

에이전틱 엔지니어링은 AI를 프로덕션 수준으로 다루는 방법론.

둘 중 하나가 아니라 — 함께 갈 때 가장 강합니다.

그리고 그 힘은 — 도메인 지식이 깊을수록 극대화됩니다.

END

감사합니다.

코딩하는 매 순간에, 좋은 vibe가 함께하기를.

Q & A

가장 동의가 안 되는 지점은 어디였나요?
반례, 이의, 감상 — 무엇이든 환영합니다.

이의 환영

반례 환영

감상 환영

APPENDIX · 그래서 다음은?

부록 — 각 진영의 전략

AGI를 향한 각 진영의 전략 — 2026년 7월 기준

⚠ 이하는 발표 시점의 관측·전망을 재구성한 시나리오입니다. 일부 모델명·수치·사건은 확정된 사실이 아닐 수 있으니 참고용으로 봐주세요.

🌐 Google DeepMind

👤 Anthropic

🇨🇳 중국 AI Labs

🌐 OpenAI

Google DeepMind — 전문가 모델 군단, 흔들리는 왕좌.

강점

전문가 모델을 찍어낸다

AlphaGo · AlphaFold · AlphaCode · AlphaProof...

20+ 년의 AI 연구 저력과 거대 연구인력 풀의 발현.

진심인 분야

Multi-Modal — Gemini 3.5

실시간 통역(Live Translate)·로보틱스(ER 1.6)는 6월 출시, 정작 Gemini 3.5 Pro는 7월로 밀림.

텍스트·이미지·영상·코드를 하나로 묶는 방향은 여전히 유효.

Tool 군단

Flow · Stitch · Jules · Antigravity

영상(Flow), UI 디자인(Stitch), 비동기 코딩(Jules), 에이전틱 IDE(Antigravity).

AI를 전방위로 확산시킨다.

반전 · 2026.06

인재 엑소더스 — 일주일 새 핵심 두 명 이탈

Gemini 공동 리더 Noam Shazeer(Transformer 저자)는 OpenAI로, AlphaFold의 John Jumper(노벨화학상)는 Anthropic으로.

Alphabet 시총 \$250B 증발 — 1년래 최악의 하루(6.22).

Anthropic — Opus의 손발을 늘리다.

전략

모델은 꾸준히, 도구는 무한 확장

Claude Sonnet 5 출시 이후에도, 실제로 할 수 있는 일을 늘리는 데 사활을 걸고 있다.
지금의 "하네스 엔지니어링" 트렌드의 시작점.

코딩

🌟 Claude Code · Cowork

CLI 에이전트 + 데스크톱 knowledge work 에이전트.

확장

🔌 Plugins · Skills · MCP

생태계로 능력을 외부에 위임. 엔터프라이즈용 Apps Gateway까지.

디자인

🎨 Fusion · Blender · Adobe

2026.04.28 — 9종 creative connector 출시.

사건 · 2026.06

정부가 모델 접근을 사흘 만에 막은 사건

탈옥(jailbreak) 우려로 미 정부가 출시 3일 만에 Fable 5·Mythos 5를 전량 차단 — 6.30 해제.
John Jumper 영입 + Anthropic·OpenAI 나란히 6월 IPO 비밀 신청.

중국 AI Labs — 개인화 Agent의 시대.

대표 트렌드

Manus 계열 — 개인화 Agent

"각자 손에 비서를 하나씩" — 개인화 에이전트가 대흥행.

해외 오픈소스 OpenClaw 열풍 (2천만 MAU · GitHub 25만 스타)과 같은 흐름.

놀라운 숫자

140조 토큰 / 일 (2026.03)

2년 전 하루 1000억 토큰에서 **1,400배** 폭증.
연산량 기준 이미 미국을 앞질렀다는 분석.

자국화

화웨이 Ascend (昇騰)

자체 AI 가속기 + 비용 절감.

NVIDIA 의존도 탈피 + 학습/추론 비용 다이어트.

최근 사건

Manus 인수 차단

Meta의 인수(\$2-3B 규모)를 중국 NDRC가 거절 (2026.04.27).

국가 차원의 AI 자산 보호 신호.

→ 5대 랩이 토큰 가격을 최대 99% 인하, 상하이 선물거래소는 AI 토큰 선물시장까지 준비 중 — "많이 · 싸게 · 모두에게".

OpenAI — 침묵을 깬 GPT-5.6.

반년 가까이 조용하던 알트만이, 6월 26일 한 번에 섯을 꺼내들었습니다.

신모델 3종

Sol · Terra · Luna (GPT-5.6)

플래그십(Sol) · 범용(Terra) · 경량(Luna).
Terra는 GPT-5.5급 성능을 절반 가격에.

이례적인 조건

정부 승인받은 20개사만 우선 접근

사이버 오남용 우려로 역대 최강 안전장치 탑재.
단계적 확대 예정 — AI도 정부 허가가 필요한 시대.

→ Cerebras 위에서 초당 750토큰(기존 대비 15배)까지 준비 중.
Gemini의 Noam Shazeer 영입 + Anthropic과 나란히 6월 IPO 비밀 신청.

그리고 — 다른 색깔의 두 진영.

빅 4가 LLM 챗봇을 두고 경쟁할 때, 이 둘은 다른 질문을 던집니다 →

"AI를 어디에 박을 것인가?"

M Meta

디바이스 확산 + 전략 선회

Ray-Ban Meta 글래스 · Quest 헤드셋 · WhatsApp / Instagram 임베드
+ **SAM 3**(영상 분할), **V-JEPA**(월드 모델).

동시에 Llama를 대체할 Muse Spark 공개 — 오픈소스에서 **폐쇄형으로 선회**.

x x.AI

현실 세계로 들어가는 AI

Grok 5 대신 **Grok Build · Voice · Imagine**을 먼저 출시, Tesla 자율주행 + Optimus 로봇.

Grok 4.5(1.5조 파라미터, V9 기반)는 비공개 베타 — Databricks 제휴로 접점도 확장.

차·로봇·PC까지 — AI를 물리 세계의 모든 표면에 박는다.

공통점 — 챗봇 자리 다툼이 아니라 → **"AI가 들어갈 디바이스" 자리 다툼.**

그럼 — Meta를 나온 안 르쿤은 뭘 하고 있을까요?

앞서 봤던 그분 — "LLM은 막다른 길"이라고 4년째 외쳐온 분.

2025.11 — Meta 퇴사

12년 만의 결별

Llama 4 실패 → Zuckerberg가 Scale AI 지분 49% 인수 (\$14.3B) + Alex Wang 영입 + Superintelligence Labs 신설.
"LLM을 더 크게 키운다" 방향으로 메타가 선회. 르쿤의 신념과 정면 충돌.

2026.03 — AMI Labs 창립

파리 · 시드 \$1.03B

Advanced Machine Intelligence Labs.

유럽 역대 최대 시드 (pre-money \$3.5B). 자기 길로 가겠다고 단단히 베팅.

기술 비전 — JEPA / World Model

"언어가 아니라 세상을 보고 배워야 한다"

텍스트 다음 단어 예측이 아니라, 영상 · 소리 · 센서 데이터로 물리 세계를 이해하는 모델을 만든다.
→ AI 산업 전체가 잘못된 방향이라는 베팅. LLM 시대 다음을 꿈꾸는 사람.